

AN ÖZYEĞİN UNIVERSITY THESIS EXAMPLE

A Thesis

by

Kaan Menekşedağ

Submitted to the
Graduate School of Sciences and Engineering
In Partial Fulfillment of the Requirements for
the Degree of

Master of Science

in the
Department of Electrical and Electronics Engineering

Özyeğin University
September 2019

Copyright © 2019 by Kaan Menekşedağ

AN ÖZYEĞİN UNIVERSITY THESIS EXAMPLE

Approved by:

Professor M. Oğuz Sunay, Advisor
Department of Electrical and
Electronics Engineering
Özyeğin University

Professor M. Reha Civanlar
Department of Electrical and
Electronics Engineering
Özyeğin University

Professor Ali Özer Ercan, Advisor
Department of Electrical and
Electronics Engineering
Özyeğin University

Professor Barış Aktemur
Department of Computer Science
Özyeğin University

Date Approved: 1 September 2012

Professor Bülent Sankur
Department of Electrical and
Electronics Engineering
Boğaziçi University

To My Family

ABSTRACT

This is where the abstract of the thesis goes.

ÖZETÇE

This is where the Turkish version of the abstract goes.

ACKNOWLEDGEMENTS

This project has been funded by Scientific and Technological Research Council of Turkey (TÜBİTAK) Project No : 117M420.

We also acknowledge the support of NVIDIA Corporation with the donation of the Titan X GPU used for this research. Partial support has also been received from the Center of Energy, Environment and Economy at Ozyegin University.

TABLE OF CONTENTS

DEDICATION	iii
ABSTRACT	iv
ÖZETÇE	v
ACKNOWLEDGEMENTS	vi
LIST OF TABLES	ix
LIST OF FIGURES	x
I INTRODUCTION AND BACKGROUND	1
1.1 Numerical Modeling of Thermal Radiation	1
1.2 Radiative exchange between gray,diffuse surfaces	3
1.2.1 Radiative Exchange Between Black Surfaces	4
1.3 Radiative Exchange Between Gray and Diffuse Surfaces	6
1.4 Monte Carlo ray Tracing Method (MCRT)	7
1.4.1 Ray Tracing Method	9
1.4.2 Ray-Surface Interaction	10
1.5 Graphical Processing Unit	11
1.5.1 CUDA and OptiX of NVIDIA	12
1.5.2 Visualization of the Numerical Simulation	16
1.6 OpenFOAM	17
1.6.1 View Factor	17
1.6.2 Radiation Calculation of OpenFOAM	19
1.6.3 Net Radiation Method of OpenFOAM	19
1.6.4 Evaluation of View Factor Radiation Model in OpenFOAM	20
1.7 Motivation of Thesis	21
1.8 Outline of Thesis	22

II	NUMERICAL METHODS	23
2.1	Ray Tracing with Monte Carlo Method	23
2.1.1	Generation of Random Numbers and Emission Points	24
2.1.2	Emission Point Generation	25
2.1.3	Launch Direction	26
2.1.4	Ray-object Intersection	26
2.2	Radiative Heat Flux Calculation with Monte Carlo ray tracing method in OptiX	27
2.2.1	Algorithm for the Monte Carlo Ray Tracing based surface – to – surface radiation model	28
2.3	View factor Calculation – OpenFOAM View factor Calculation Utility	29
III	RESULTS	31
3.1	OpenFOAM and Monte Carlo Method View factor Calculation Utility Results	31
3.2	Transient Conjugate Heat Transfer Scenario	38
IV	CONCLUSION	45
V	ACKNOWLEDGEMENT	47
APPENDIX A	— SOME ANCILLARY STUFF	48
REFERENCES		49
VITA		50

LIST OF TABLES

1	NVIDIA Titan Xp Graphics Card	13
2	Computation Time for Surface-to-Surface Radiation Model in Open-FOAM	36
3	Computation Time for Heat Flux using Monte Carlo Ray Tracing on Graphic Card	37
4	Computational Cost for Radiation Heat Transfer in OpenFOAM using 1 CPU	40

LIST OF FIGURES

2	black enclosure geometry	4
3	blackbody radiation	5
4	blackbody net radiative energy exchange	5
5	Ray Tracing Algorithm Fundamental	9
6	Ray Tracing concept is simple but computational cost is too much . .	15
7	Traversal part is controlled by the OptiX. Developers can change programmable shaders (green) in OptiX.	16
8	Figure 2(a):	18
9	Figure 2(a):	18
10	Figure 2(a):	18
11	Figure 2(a):	18
12	Figure 2(a):	19
13	Algorithm for the Monte Carlo Ray Tracing based surface – to – surface radiation model	29
14	F_{ij} exchange matrix calculation from each i surface to j surface . . .	30
15	View factor calculations between the two planes (calculation was made by OpenFOAM software with deterministic ray tracing method. . . .	32
16	Calculation of Radiative Heat Flux btw surfaces : Cell Size vs. Accuracy	33
17	Figure 1: View factor calculations between the two planes (Left calculation was made by OpenFOAM software with deterministic ray tracing method. The right calculation was made by newly developed software with Monte Carlo ray tracing (MCRT) method.)	34
18		35
19		37
20		38
21		39
22	View Factor between the glass and the heater (Left).Standard deviation in view factor calculation (Right)	41
23	Total computation time comparison	42

24	Total Computation Time for using Multiple CPUs and single GPU .	43
25		44

CHAPTER I

INTRODUCTION AND BACKGROUND

One of the most significant modes of heat transfer is the thermal radiation in today's high temperature engineering solutions. The engines, solar collectors, furnaces, combustors are the highly specific areas that require special design and analysis of the thermal energy conversion processes. The thermodynamic efficiency of the high temperature radiation dominant processes is depended on the ratio to deal with radiation effect. Thermal radiation is able to propagate through space without any need of medium and the intensity change is governed by the Radiative Transfer Equation (RTE) which is an integro-differential equation expressed in terms of six parameters. Howell et al. [1] has been discussed a comprehensive derivation of the radiative heat transfer equation. RTE can be defined in differential form as following EQ1:

If Intensity/area=differential of flux/area or Intensity/area=single ray* F_{ij} *areanetradiationmen
that the term of the time rate of change in spectral intensity per unit projected area is expressed on the left hand side and the terms of the effects of emission, absorption and scattering of the intensity of light are at the right hand side of the equation.

1.1 Numerical Modeling of Thermal Radiation

The Radiative Transfer Equation can be solved numerically only with the exception of some simplified scenarios. Various types of numerical methods of RTE have been developed and applied to different type of problems. These can be splitted into two major classifications based on how it interacts with the medium. One of them is that thermal radiation penetrates through medium which is participating. Under these circumstances, different numerical methods are used such as discrete ordinate method [], zonal method [], spherical method, finite volume method [] or Monte Carlo ray

tracing method [1]. In addition to this, the radiation heat transfer problems where medium is neglecting is considered as interacting with the surfaces of the geometries. For this purpose, calculation of the heat transfer is based on the so-called view factor. It is calculated between surfaces of the different object geometries. However, it is possible to calculate analytically only when the geometries are standard shapes. More detailed versions of several view factor calculations are carried out by Howell and Menguc [2]. On the other hand, not all the geometries have regular shapes in this world. View factor should be conducted numerically for the complicated scenarios which is most of the radiation cases. To be able to calculate the view factor, mostly used ones are Cross-string Method by Hottel [3], Optical Projection Method by Farell [4], Hemi-Cube Method by Cohen [5], Unit Sphere Method by Nusselt [6] and Monte Carlo Ray Tracing Method [7].

In particular, a numerical method is expected to be precise as well as computationally quick. However these two demands appear to be conflicting, computationally faster solution can also provide more accurate result than computationally expensive solution according to the problem types. For instance, even the simplest version of P1 the Spherical Harmonics method may solve more accurate solution compared to the Monte Carlo Ray Tracing method which is accepted as computationally the most expensive and accurate method. Likewise, the Discrete Ordinate method is used scenarios where non-participating medium is involved may cost too much time and power while the precision is not satisfied due to observed errors in the solution as reviewed methods by the Chai, Lee, Patankar [8].

Monte Carlo Ray Tracing method is known as the most accurate for complex scenarios among all the other methods in literature which is stated by Howell [9]. A semi-analytic approach to solve the integro-differential equation for the problems of thermal radiation. Concerning the studies of the view factor, Emery et al. [16] has been carried out comparisons between unit sphere method by Nusselts, Cross-string

method by Hottel, optical projection method by Farrell and Monte Carlo ray tracing method for the calculations of diffuse view factors. As a result of this investigation, Monte Carlo ray tracing method has been demonstrated significant excellence regarding to accuracy of calculations. The research of view factor calculation conducted by Mirhosseini and Saboonchi [1] has illustrated the MCRT method's accuracy between strip components and cylinder. Hajji et al. [2] executed a similar research and figured out the improved accuracy of the MCRT technique compared to the the Cross-string method between semi cylinder and fins. Moreover, scenarios containing participating medium have evaluated using various numerical methods. The study by Demirkaya [3] investigated the accuracy of Monte Carlo ray tracing and discrete ordinate method in cases involving scattering, absorbing and emitting media using three dimensional geometries. The Monte Carlo ray tracing method presents more accurate results for the scenario of the research. There is also another study carried out by the Henson et al. [4] and it also provides similar outcome in terms of accuracy which is Monte Carlo method is better than discrete ordinate method.

1.2 Radiative exchange between gray,diffuse surfaces

Emitters,absorbers, integralli ya da numerik Section:Surface Radiative Heat Flux Calculation olabilir

Radiative exchange without participating medium within enclosure can be calculated. In general cases, it is using the view factor exchange matrix. But ,in this study, the developed approach is to calculate radiation directly without view factor. Radiative heat transfer of the black enclosure can be easily evaluated because rays will be only absorbed or missed on the black surfaces.

Enclosures like furnace problems are simple to examine sufficiently due to black covered walls. Radiation is absorbed on black surface regardless of light characteristics. Light normally travels with a certain payload depending on the wavelength.

There are also grey or diffuse surfaces changing the radiation amount earning. Radiative properties of diffuse surfaces does not depend on the wavelength while it is possible to emit or reflect diffusely. Most of the surfaces in the nature emits or absorbs energy bundles between 60° to perpendicular to surface normal. Remaining part of the light propagation can be neglectable for heat transfer calculations. The vast majority of the materials are tend to have diffusive feature. Even in a small change in the spectrum, it can lead the properties turn into black body surface features. It is feasible to simplify grey surfaces.

Fistly, governing equations should be derived for the enclosure cases no matter what the types of the surfaces are, black or grey, diffuse surfaces. It is easier to solve a set of arrange equations. Normally integral equations are applicable all sorts of random enclosures.

1.2.1 Radiative Exchange Between Black Surfaces

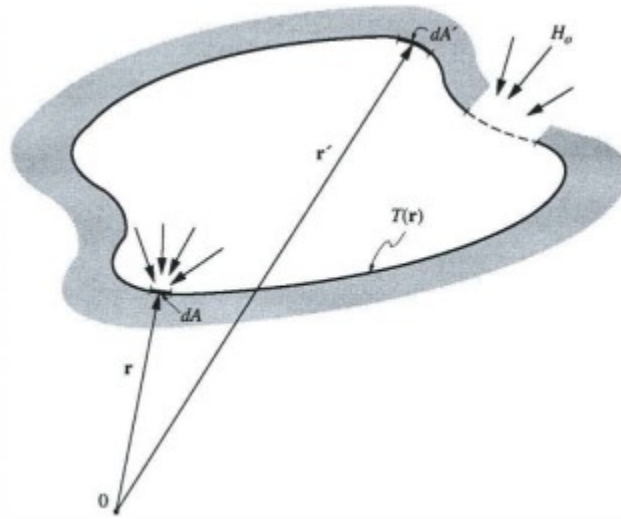


FIGURE 5-1
A black enclosure of arbitrary geometry.

Figure 2: black enclosure geometry

A nonuniform shape of black-surfaced enclosure with non-homogeneous temperature distribution as Fig 2 can be evaluated. Bundle of rays can be absorbed or emitted

energy from infinitesimal area of dA. Energy exchange for dA in total can be explained as equation 1

$$q(\mathbf{r}) = E_b(\mathbf{r}) - H(\mathbf{r}) \quad (1)$$

where E_b is the intercepted energy rate from dA' to dA and H is the general irradiation from surrounding to dA. Total heat transfer energy rate from whole enclosure and outside sources from holes or semi-transparent surfaces combined occurs the H(r).

The heat transfer problem can be expressed as dividing it into small areas and solve them independently across the entire enclosure surfaces. Irradiation can be counted as merging subsurfaces. Heat flux is calculated using local view factors F_{di-j} because it can vary and become nonuniform. Average heat flux can be defined in simplified version as $A_i F_{i-j}$. It yields equation substituted with E_b in Eq 3

$$q_i = \sum_{j=1}^N F_{i-j}(E_{bi} - E_{bj}) - H_{oi}, \quad i = 1, 2, \dots, N.$$

Figure 3: blackbody radiation

The equation expresses the heat flux as net radiative energy exchange between surfaces A_i to A_j .

$$Q_{i-j} = q_{i-j}A_i = A_i F_{i-j}(E_{bi} - E_{bj}) = -Q_{j-i}.$$

Figure 4: blackbody net radiative energy exchange

The net radiative heat transfer between dA_i and dA_j is written in the formula ()

$$\dot{Q}_{A_i-A_j} = A_i F_{A_i-A_j} \epsilon \sigma (T_i^4 - T_j^4) \quad (2)$$

where the net heat flux is $\dot{Q}$, area of the surface i is A_i , Stefan Boltzmann constant is σ , emissivity of the A_i surface and the temperatures of the surfaces T_i and T_j .

1.3 Radiative Exchange Between Gray and Diffuse Surfaces

In the gray surfaces, all materials are accepted as diffuse absorber, emitter, reflector. Surface radiosity "J" is the total heat flux sourcing from point $\mathbf{r}$. Eq 3 shows the total energy rate summation of both the emitting and reflecting from point $\mathbf{r}$.

$$J(\mathbf{r}) = \epsilon(\mathbf{r})E_b(\mathbf{r}) + \rho(\mathbf{r})H(\mathbf{r}) \quad (3)$$

It is very hard to differentiate how much portion of the radiation occurs due to emission or reflecting energy. One can analyze the rays just looking at temperatures T_1 and T_2 . Each ray will have different wavelength carrying the energy payload with different temperatures. It can be emitted from inner blackbody surface with temperature T_1 and reflected ray generated from outer blackbody surface with temperature T_2 . If someone wants to know the type of rays, features of the ray beam shows it as emitted or reflected. However, a gray surface is lack of detecting this rays and distinguish the types. Each ray is accepting as incoming radiation no matter what type the wavelength is. It is called as color blind. Source of the irradiation is unknown, it can arrive with emissive power J from black or gray,diffuse surface. It helps to calculate surface radiosity in simplified version. Balanced surface to surface net radiation is combination of net incoming (absorbed) and outgoing radiation (emitted or reflected from surface to any other surface). In the literature, it is referred as net radiation method due to balancing energy as shown in Eq 4.

$$q(\mathbf{r}) = \epsilon(\mathbf{r})E_b(\mathbf{r}) - \alpha(\mathbf{r})H(\mathbf{r}) = J(\mathbf{r}) - H(\mathbf{r}) \quad (4)$$

To solve the governing equation, irradiation for the infinite-small areas dA and dA' are only pieces of understanding the solution. Governing integral equation to solve the radiation is the overall heat flux calculation.

1.4 Monte Carlo ray Tracing Method (MCRT)

The Monte Carlo ray tracing method is a stochastic approach to solve an integro-differential equation based on the random sampling to deliver numerical outcomes. Number of trials determines the accuracy of the method. In particular, the accuracy gets better as the number of samples increases. Monte Carlo ray tracing method is very popular nowadays owing to development of the advanced computers that supplies huge amount of computing power. Thanks to this, the widespread range of areas of engineering and science can take advantage of the Monte Carlo method.

Monte carlo method is very suitable to solve various complex problems such as radiation heat transfer or other modes of the heat transfer. Howell and Perlmutter [] has addressed the implementation of Monte Carlo method for radiative transfer equation solution. the RTE of the scenario of two parallel plates with participating medium between them is solved. Usiskin and Sparrow [] has validated the devised method by comparing with analytical solution. Complicated radiation heat transfer problems is solved effortlessly by implementing Monte Carlo method. Howell [] has mentioned in the study that Monte Carlo method has enormous advantages over all the other numerical methods with regard to accuracy. The analysis of precision of cases involving complex geometries, spectral properties and inhomogeneous media benefits from random sampling using high number of trials. The challenges can be handled easily, effectively with flexible way using the Monte Carlo method. There is also another advantage of the nature of randomness of Monte Carlo method in terms of speed up by processing problems on parallel cores or devices. There have also been numerous changes to the conventional Monte Carlo method. Haji-Sheikh [] has developed the method solving the problem both conduction, convection and radiation. As a matter of fact that RTE can be solved easily and ideally with Monte Carlo ray tracing method due to the feature of representing the nature of light. In ray tracing concept, every ray propagates through a straight direction unless it interferes

with any object. The technique allows tracing multiple number of rays where they behaves like photon bundles which contains certain amount of radiative energy. The photons travel the path defined by random directions depending on the medium's radiative properties. In the life span of photon package, a natural phenomena may occur like emission, absorption, reflecting or scattering. Modest [] has discussed all the possible events in detail.

Despite the fact that Monte Carlo method is beneficial in terms of precision, the method is avoided to apply on the scenarios because of its computational cost. This problem prevents it from being used widely. On the other hand, today's modern technology makes available to overcome the obstacle using computationally expensive Monte Carlo method. Cutting-edge multi-core processors and many-core architecture of graphic accelerators are ideally suited for the high performance accurate solutions. Loop et al. [] has discussed that graphic processor unit (GPU) was created specifically to help the rendering tasks which are computationally costly. Over the years, scientific computing has been expanded to utilize code libraries because of significant increase of GPU's computation power. The modeling of the solar radiation of urban area that has analyzed using GPU and the Optix SDK of Nvidia is explained in the study by Halverson []. He et al. [] used graphic processors to solve radiation heat transfer in the problems of propulsion and combustion. The surface-to-surface radiative heat transfer for complicated three-dimensional geometries is able to solve using GPU by Siddiqui et al. []. The graphic processor radiosity method was introduced by Takizawa et al. []. The high performance of GPU's attracts researchers to utilize parallelism structure for accelerating their computations. In addition, discrete ordinate method is used to solve one-dimensional radiative transfer equation comparing the results of multi-processor CPU's and GPU's by the Efremenko et al. []. With the benefits of GPU, the process has accelerated and 50x speed-up has accomplished.

1.4.1 Ray Tracing Method

Ray tracing is a method designed in computer graphics for rendering images. Nowadays, it is currently used frequently to produce realistic graphics from CAD designs. It also reveals opportunity to calculate realistic light effects over object surfaces. In this method, a virtual camera eye is viewing spot to produce the three-dimensional image of object. Each pixel is a emitting point where the rays starts to travel through domain and rays are traced unless they intersects with an object in screen. The color to be displayed on the screen is determined by the ray-object intersection case. Figure shows the fundamental principle of the ray tracing method.

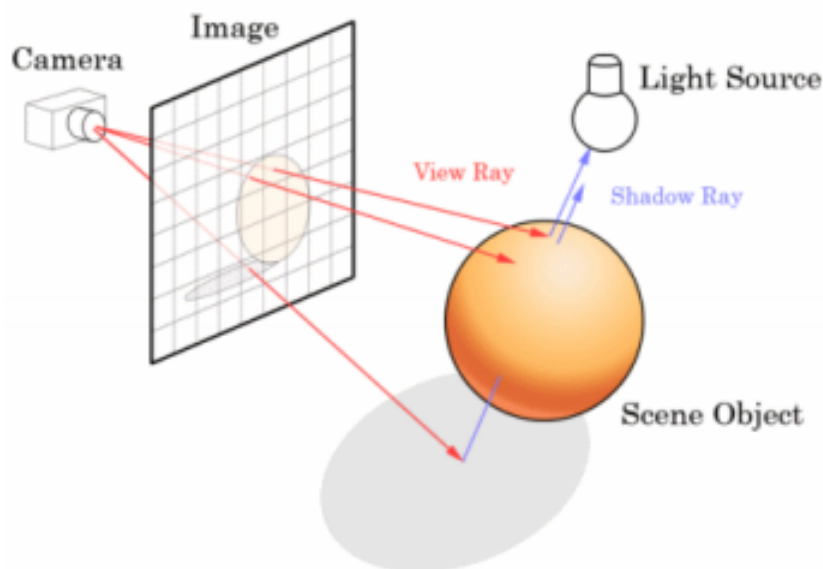


Figure 5: Ray Tracing Algorithm Fundamental

Primary rays are the ones traced from the camera to the scene object. They are the initial rays triggering more scatter rays after hitting the object. New set of iterated rays can be traced with different payloads effects while they penetrates in the domain like the original rays. Secondary rays such as shadow rays can determine the heterogeneous scene illumination. It allows to design the light sources behaving on transparent materials whether the object is reflecting or absorbing. There are hit

rays, reflected rays, absorbed rays and the missed rays. Each ray has limitations for the time or distance during its travel. On the display screen, every pixel is arranged the colour and texture according to hit or miss rays. After reaching a boundary wall or maximum iteration number. Screen colors of the pixels are set when the all rays are traced and finalize as hit or missed.

Appell [1] has first developed the idea of ray tracing to the computer graphics area in 1980 when he produced three-dimensional model along with the impacts of reflection and refraction. After that, Kajiya [2] has launched merged ray tracing algorithm with Monte Carlo method and showed the randomness in the ray path to analyze the impact of reflected light on the scene. Although the method of ray tracing can produce quite realistic scenes, it is known that using the method for rendering images is computationally more costly than other methods. Ray tracing method requires huge computation cost and is slow even for the moderate complex meshes. On the other hand, computation power of today's modern computers increases significantly and this gives great opportunity to trace rays even in real time applications. In the coming years, Monte Carlo ray tracing method is capable of the most preferred method for rendering images in the field of computer graphics.

1.4.2 Ray-Surface Interaction

Ray tracing means that a simulated artificial ray from a source point releases and travels through domain until interacts with a geometry. The algorithm follows the line of sight principle which supports a point-to-point 3D line. Advantage of the point-to-point process is that suitable for all different shape geometric objects. For instance, calculation on a sphere can be done easily without tessellation mesh. Cubes, cylinders, triangle prisms are the general shapes in usage. On the other hand, triangle primitives are the most useful shape because almost all complex geometries can be defined by triangles.

Intersection point can be calculated using by Eq 5. A ray has a origin point "o" and tracing starts from that point in the direction vector of energy packet "d". Position of the intersected point "p" is reached by time "t" the ray travels until hit point. In this perspective, intersection point is highly depended on the time "t" value since the origin and direction of the ray are prerequisites for ray tracing

$$\mathbf{p}(t) = t\mathbf{d} + \mathbf{o} \quad (5)$$

The eye from pinhole camera follows the path using Eq. For the secondary ray, origin point replaces with the intersection point of the primary ray and a new direction vector is assigned.

1.5 Graphical Processing Unit

The Graphical processing unit (GPU) is a customized electronic circuit completely developed for computer-based rendering applications. "The World's First GPU" [] was the motto of launching the GeForce 256 for computer industry in 1999. GPU's which designed to operate parallel computations are the preferred and perfectly suited for rendering images. The reason behind this is that CPU is utilized for latency while the GPU is designed for throughput. The capacity of CPU is limited only dozens of processing cores though the GPU has thousands of cores per chip. It includes multiple processors and each one of them is including huge number of thread processors which can concurrently perform computations. However, CPUs will remain as the main part of the computer since dependency of serial calculations are will always exist. Today's technology gives the opportunity to avoid performing each calculation in the serial pipeline. Parallel or serial computing concepts are not meaningful by their own. Hybrid computational solutions are more effective in time.

The demands of the computer graphics industry lead the way strengthening parallel power of GPU on the contrary of CPU. Apart from field of computer graphics,

other computation requirements from scientific computing tries to take advantage of parallel processing power of GPU to speed-up their performance. Considering GPU-accelerated computing, efficiency of computational process of solving mathematical formulas boosted extremely and it attracts suitable applications to be calculated on GPU's. The combined usage of GPU and CPU is a efficient way to accelerate solutions of scientific computing. GPU-accelerated computing seperates the computationally expensive load to GPU side while the remaining part of the load to CPU for processing. Application of LU factorization [1] became the lead study where solving a scientific problem is faster than calculating on a CPU. Various applications in a wide range of areas such as video processing [2], computer vision [3], molecular modelling [4], fluid dynamics [5], bioinformatics [6] and even astrophysics [7] have been extensively benefited from GPU's.

1.5.1 CUDA and OptiX of NVIDIA

The performance of computation is strongly dependent on the how it is coded to run on the GPU hardware. Programming the graphics card is getting important every day because numerous devices from tablets to cell phones, personal computers to super computers are using advanced GPU's developing computation capabilities. In order to utilize graphic cards for users who wish to create their projects, several softwares and libraries have been developed throughout the years such as CUDA, Optix, OpenCV, OpenGL or OpenCL. It is not only the rendering process that is utilized the computational ability of graphic cards but all the frameworks that has mentioned provide optimizing the code that runs parallel processing applications on GPU's for much more effectively and results in significant speed-up.

CUDA is developed by Nvidia as application programming platform (API) computing parallel for general purposes. It is a framework that allows developers benefits from CUDA-enabled graphics processing unit and to build any application which is

possible to be processed on GPU [1]. Utilizing parallel processing power of GPU to accelerate the applications is a groundbreaking innovation in the field of science and engineering.

The NVIDIA Titan Xp has been used for the research to accomplish the high performance computation. Table 1 shows the specifications of the card.

Table 1: NVIDIA Titan Xp Graphics Card

	NVIDIA TITAN Xp
NVIDIA CUDA® Cores	3840
Memory Speed	11.4 Gbps
Standard Memory Config	12 GB GDDR5X
Boost Clock	1.6 GHz

Nvidia OptiX Software Development Kit (SDK) is specially designed on CUDA to develop ray tracing applications [2]. The power of Nvidia GPU's offers multiple functions of programmable intersection of the scene and maximizing various light techniques on complex objects. The various methods of acceleration are possible to enhance the performance of application in the context of the ray-object relation. The Optix SDK's ultimate objective is making the best of graphics processing unit and improve the performance as much as possible. The target market is not only limited with game industry, the aim is that ensuring necessary tools for every user who requires to develop ray tracing applications to use OptiX. All softwares using CUDA enabled GPU architecture can get benefit from OptiX SDK.

The OptiX engine focuses on the executing the ray tracing algorithm on the programmable set of processes. OptiX also has been developed suitable to render real-time 3D images like OpenGL. OpenGL has a stage called rasterization which pixels are covered by triangle mesh and interpolate the pixels according to vector image. OptiX is applying this procedure by the power of the parallel programmable rasterization

pipelines. OptiX has been architected to compile the domain of ray tracing in multiple threads. Designed kernels are traced the rays of the user's developed software to respect to geometric shapes and limitations. Object intersection, material shading and ray production are the main processes of the ray tracing engine. The OptiX has huge advantages in terms of using parallelism on the different applications such as real-time rendering, offline rendering, illumination of the environment, artificial intelligence, scientific real-time software simulations like radiation heat transfer.

OptiX has customize in-house built object model to decrease the amount of data of scheduled tasks. The OptiX engine is able to handle general commands unlike the OpenGL restrictions of fixed data structure or rendering. The SDK provides flexibility of quick implementation of the ray tracing project. The developed program can be easily implemented on the general purpose GPU and the computationally heavy tasks for the CPU can be run fastly and efficiently even on the standard GPU.

The OptiX algorithm have multiple major steps for the ray tracing. Every programmable part has unique accessibility to different regions of the OptiX. Graphic card is enabled some kernels according to the interruption or invocation while executing at some critical levels of the ray tracing algorithm. Ray tracing pipeline on the OptiX kernel is shown in the Fig. There are multiple call functions to be able to execute ray generation program in desired way. The center of the process graph is dominated by "rtTrace" which controls or get feedback from peripheral functions. These functions are representing source side intersection (traverse) and destination side intersection (shading). All the processes of the these functions allocate global and local memory, costs time for reading or writing data to payload of the ray.

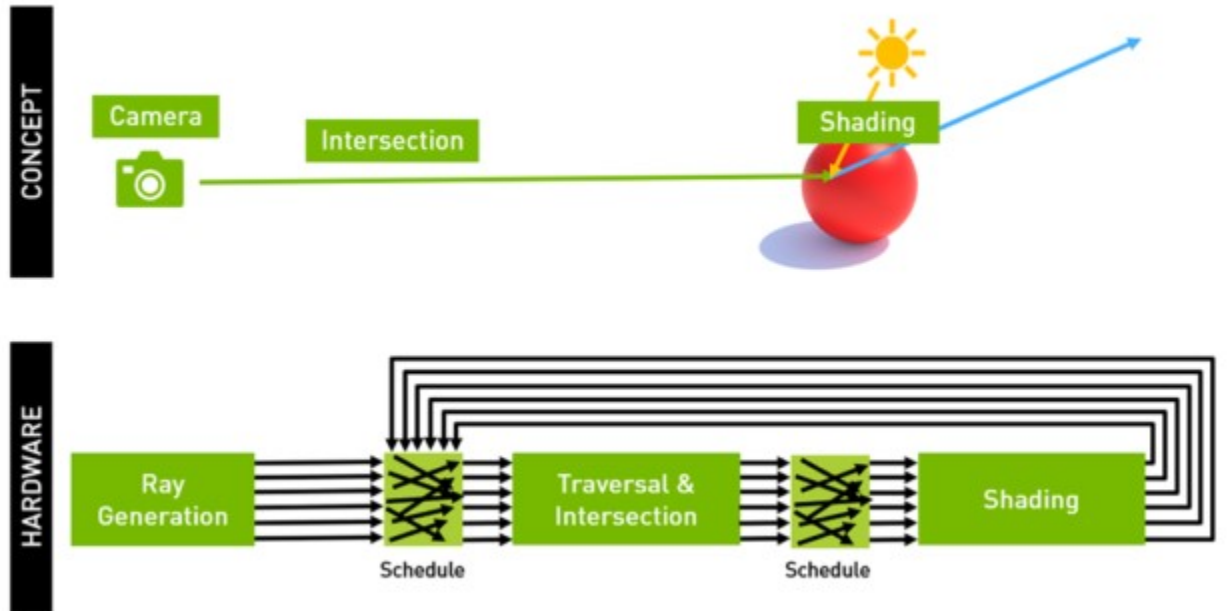


Figure 6: Ray Tracing concept is simple but computational cost is too much

OptiX API is developed for the bridge the gap between the conceptual model and roughly enhanced execution model on GPU. Developers are enabled to concentrate on their main ray tracing algorithm rather than the scheduling their code. OptiX is a platform for whom needs a ray tracing algorithm executing numerous the programmable operations. The engine gives opportunity to create a virtual environment and every ray-tracing step in the cycle has benefit from programmable shaders such as miss, any hit or closest hit.

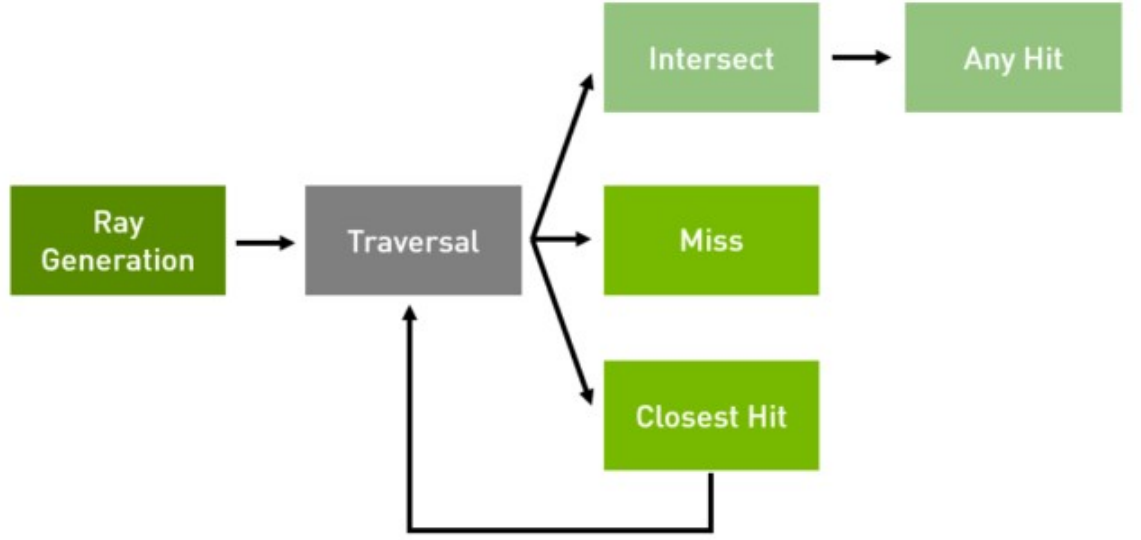


Figure 7: Traversal part is controlled by the OptiX. Developers can change programmable shaders (green) in OptiX.

Ray generation is firstly responsible to trace the primary ray trough scene, transfer the data to output buffers. After pre-procedure of generation has done, the ray traversal acceleration part is selected. OptiX main compiler makes generation and data structure traverse much more easier since it has internally controlled. Efficient solution of acceleration structure into the OptiX hides away the hardness of problem, special data architectural implementations.

1.5.2 Visualization of the Numerical Simulation

In numerical computation, visualization is separated from solving the problem and considered as post-processing. Combination of solver and visualization of the numerical simulation is a popular concept and brings several advantages which are monitoring real-time and verify the the results as computations are running. In this research, the outcomes of simultaneous computation and visualization is planned to execute by developed software using this integration approach. A user is allowed to control and analyze progress in a three-dimensional scene.

The developed software using the Optix environment delivers the solution of RTE tracing tracing rays in the domain. Simultaneously, the three-dimensional objects are highlighted as a color representing the calculation results or a static situation.

1.6 *OpenFOAM*

The widespread used CFD softwares today are the ones require expensive annual or permanent license. They are pocket, ready-to-use programs which allows limited access to modify solvers, parameters, geometries and constrained by cost of multi-user or multi-processor. However, OpenFOAM is a free open-source CFD software is powered by a huge community from most fields of engineering ,science , academic organizations and commercial companies. OpenFOAM has a wide variety of features for arranging which solvers, parameters is going to be used to solve areas such as heat transfer, turbulence, acoustics, solid mechanics, electromagnetics or complex fluid flows. New version of the open-source CFD toolbox OpenFOAM of ESI-Group is released every six months by the help of customer endorsed contributions and developments from community. The software is approved itself by the wide range of usage of the development partners, engineering companies and the academia. In this thesis, all the developments and modifications are applied to the OpenFOAM v1612+ version.

1.6.1 View Factor

View factor radiation model is formed for the radiation heat transfer exchange on grey-diffuse surfaces. In this model, radiation heat transfer is not affected by the medium but only radiating surfaces are considered. Distance, orientation, separation and size of the surfaces are the parameters that hugely affects the radiation exchange. They are accounted for calculation of the view factor which is the fraction of radiation emitted from one surface that is absorbed by other surface. Radiation heat transfer is governed by the view factors between the surfaces. It heavily depends on the geometries and

positions of the objects relative to each other.

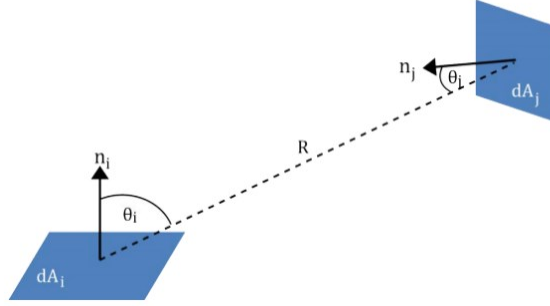


Figure 8: Figure 2(a):

The view factor between these two infinitesimal areas can be calculated as in EQ

$$dF_{dA_i-dA_j} = \frac{\cos \theta_i \cos \theta_j}{\pi S^2} dA_j.$$

Figure 9: Figure 2(a):

View factor between infinitesimal A_i and surface A_j can be formalized as shown in Figure

$$F_{dA_i-A_j} = \int_{A_j} \frac{\cos \theta_i \cos \theta_j}{\pi S^2} dA_j,$$

Figure 10: Figure 2(a):

If both the geometries are surfaces then equation transforms and view factor follows as

$$F_{A1-A2} = \int_{A1} \int_{A2} \frac{\cos(\theta_1)\cos(\theta_2)}{\pi S^2} dA_2 dA_1$$

Figure 11: Figure 2(a):

Switching i and j subscripts in equation directs to important law of reciprocity.

$$dA_i dF_{dA_i-dA_j} = dA_j dF_{dA_j-dA_i}.$$

Figure 12: Figure 2(a):

1.6.2 Radiation Calculation of OpenFOAM

Heat transfer through radiation calculation of OpenFOAM utilizes from several methods. Pros and cons of the different radiation heat transfer models are discussed with detailed look.

Computation of view factor radiation model in OpenFOAM has several steps to evaluate radiative heat flux. The model firstly initializes the object type variables reading files of the global faces, total number of faces, boundary meshes and view factor exchange matrix.

1.6.3 Net Radiation Method of OpenFOAM

Radiative heat exchange is hard to follow due to reflections, rereflections, partial absorption. Net Radiation Method is calculating radiative heat fluxes between diffusely emitting and reflecting surfaces. Heat flow is not only between two surfaces that see each other but a full face network combination of faces interacting with all the other faces. The problem is solved using set of linear equations which includes temperatures, radiative heat flux, surface properties, full specifications of geometry. Main point of calculating radiative heat exchange is the view factor between object surfaces. Position and shape is dominant factors deciding the ratio of heat transfer. Moreover, emissivity of the surfaces also determines amount of the net radiation on the face of boundary cells. Emissive power is the rate of energy emitted per unit area.

$$\begin{aligned} Cq_R &= \sigma F(T_i^4 - T_j^4) \\ e_b &= \epsilon \times \sigma \times T^4 \end{aligned} \tag{6}$$

Finally, a linear system of equations is formed that relates the radiative heat fluxes with the temperatures of the radiating surfaces.

$$Cq_R = (I - F)e_b \quad (7)$$

Where

C: Coefficient Matrix

q_R: Radiative Heat Flux

I: Identity Matrix

F: View Factor Exchange Matrix

eb: Emissivity Power

1.6.4 Evaluation of View Factor Radiation Model in OpenFOAM

The view factor is calculated solving a mathematical equation which represents radiative heat transfer contribution. This model does not take no radiation emission, absorption or scattering into account. The number of faces that radiates rays carrying energy designates the computational cost of view factor radiation model in terms of memory and time.

OpenFOAM have several radiation models for radiative heat transfer such as P1 model or finite volume discrete ordinate method (FvDOM). Besides this, the view factor model of OpenFOAM is calculating explicitly surface to surface radiation energy exchange. Boundary file of OpenFOAM mesh includes the type of the boundary surfaces. There should be written "viewFactorsWall" in which surfaces wanted to join the radiation heat transfer computation. Firstly, the faceAgglomeration utility of OpenFOAM generates the view factor wall surfaces for view Factor calculation which the viewFactorsGen utility of OpenFOAM will use in next step of the radiation model. One of the parameters in viewFactorsDict file of OpenFOAM is "nFacesInCoarsestLevel" which is limiting the number of produced surfaces. All the surfaces

are accepted as view factor surfaces if the `nFacesInCoarsestLevel` is much higher than the total number of `viewFactorWall` surfaces. Otherwise, some of the surfaces are grouping together according to the "featureAngle" parameter between face normals of surfaces. Multiple surfaces are combined to a patch if the difference of normal angles between them is smaller. View factor is calculated for this patch and the same value of heat flux is distributed to all the surfaces formed the patch. The process may fail because values of parameters in `viewFactorsDict` file is not sufficient to agglomerate all the faces. For the scenarios with huge number of faces in mesh, every command produces text files which consumes too much memory in hard disk and takes too much reading time when they are needed.

Secondly, the `viewFactorsGen` utility of OpenFOAM is operated the ray tracing between the distinct faces of the `viewFactorWall` surfaces. Each ray are traced until destination surface and radiative heat transfer is calculated by the adding every energy exchange between source and destination points. The faces of surfaces which are apparent to each other without any blocking elements in whole domain exchange their energies by this approach. It is very crucial to obtain correct solution in scenarios involving complex radiative problems that some geometries are shielded by the others.

1.7 Motivation of Thesis

The most accurate and flexible numeric method to calculate radiation heat transfer on realistic scenarios with complex geometries is accepted as Monte Carlo ray tracing method. However, it is not widely used due to expensive computational cost and researchers avoid to apply it on their test cases. The developed algorithm relies on improving usage of Monte Carlo ray tracing method and this increases the possibility to use as built-in function of next-gen heat transfer solvers. GPU-accelerated applications contains both programming level data structures and hardware level graphic processors. The present application developed in the research using Monte Carlo ray

tracing is adjusted according to coupling energy equation and RTE.

Apart from this, OpenFOAM's surface-to-surface radiation model is rather limited to simplified scenarios. Even though it is applicable to simple general cases, the radiation models of OpenFOAM requires huge memory because view factor matrix F_{ij} is not sparse. Using direct methods for the solution of linear systems are computationally very expensive.

The present developed method makes it possible to solve RTE of complex geometries accurately even in the critical regions. It is feasible for any scenario without requirement of huge memory usage of the hardware. The new method also delivers the optimized precise solution with the speed-up in calculation.

The main objective is to transform a solver in OpenFOAM to be able to deal with a scenario involving thermal radiation with Conjugate Heat Transfer. Radiation model will be replaced by more accurate and fast Monte Carlo ray tracing method in Optix. Combination of both OpenFOAM and Optix solvers will provide more correct results even in the complex geometries with satisfied speed-up in simulation time.

1.8 Outline of Thesis

The outline of the thesis starts with Chapter I which is the introduction of the research including literature review of the field. Chapter II involves numerical methods, integration of them in developed solver and execution to validation cases. Chapter III discusses the test scenarios and results of OpenFOAM and the present method. At last, Chapter IV is about the conclusion.

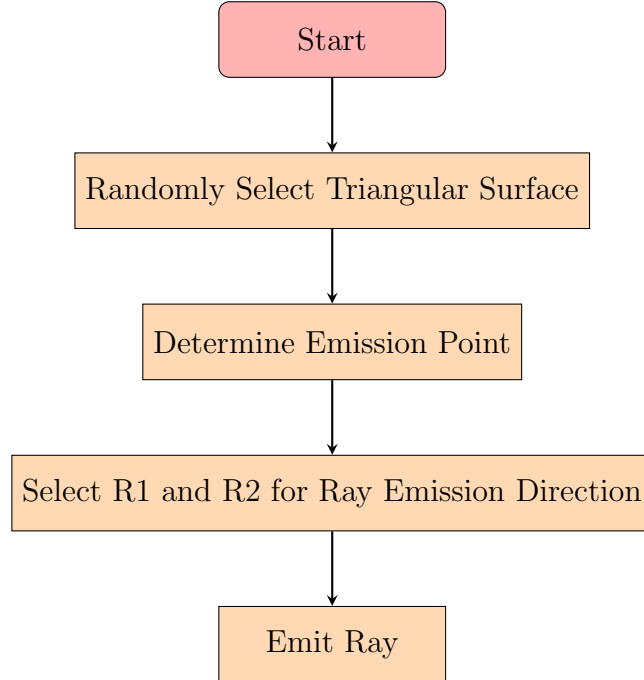
CHAPTER II

NUMERICAL METHODS

2.1 Ray Tracing with Monte Carlo Method

Monte Carlo Ray Tracing Method is used for the developed tool to solve surface-to-surface radiative transfer equation (RTE) between the surfaces of three-dimensional complex geometries. Flow chart of the ray tracing using Monte Carlo method can be seen in Figure 2.1.

Flow Chart of the Ray Tracing of Radiative Heat Flux Calculation



Firstly, triangular mesh designed by a CAD program is required to process radiation and display the calculation results on top of itself. Ray tracing of radiative heat flux calculation between discrete surfaces follows the steps in flow chart. Each geometry object file includes multiple triangles defining the boundary of the solid material. Selection among the triangular surfaces occurs according to emitting number of rays

and launch index. Each surface is fairly the source primitive of an equal number of ray tracing.

Secondary, emission point decision from the selected primitive is takes placed. It is the fundamental step to apply Monte Carlo method. Tracing a ray involves source point, direction and payload. After releasing to space domain, it will be named as hit ray if it reaches to an intersection, reflected ray if some of its energy continues to travel in another direction or miss ray if there is no intersection. Randomness of point and direction selection is the core part of present ray tracing algorithm. Extended explanation of its usage will be in the next section.

Thirdly, selection for ray emission direction is also random process. Each triangular surface has already a defined surface normal. A new random ray direction vector considering the geometrical limitations should be calculated after emission point is decided. In 3-dimension, direction vector is the combination of random numbers and some trigonometric multiplications. More detailed explanation will be in next part of launch direction.

After all preparations, each necessary parameter to emit a ray is ready for Monte Carlo ray tracing method. Inside an general purpose GPU, a thread will be assigned just for this calculation running independently from other ray tracing trials.

2.1.1 Generation of Random Numbers and Emission Points

The key problem when using the MCRT method is random numbers generation which directly affects the accuracy of the radiation model. Random numbers are uniformly produced in the current work. They are highly likely to be chosen in controlled random distributions. Limitations of the randomness is described by combination of the index of launch ray, screen size and various random numbers.

Various algorithms for uniform numbers generation are available in the literature. They can provide good but different results from each other to solve the radiation

problem such as center-to-center or variance reduction. One of them is Tiny encryption algorithm (TEA) which includes cross multiplication and summation of the some variables. Simplicity of its application and shorter code dimension than other algorithms makes TEA considerable in the limited embedded systems. It maximizes the the process speed and minimizes the memory allocation. The algorithm is applied to find radiative heat flux in this study. The probabilistic algorithm produces pseudo-random numbers by generating a seed using an cryptographic function. It is hugely accepted as benchmark to analyze the genetic algorithm. Zafar[] has found Tiny Encryption Algorithm is easy to use and scale for computation intensive applications via Monte Carlo sampling on GPU. Randomness of the emission point selection is based on variables that produced using the TEA. This are generated from variables derived from random variable.

2.1.2 Emission Point Generation

Barycentric

The accuracy of Monte Carlo method is also influenced by determining the positions of the emission point. Every point is tried to determine by equal probability on the object geometry which is split into several triangular surfaces. Each mesh surface is transformed to point cloud which are sampled uniformly. To make it happen, number of rays to be sent must be determined at the beginning of the simulation.

$$d = (1 - \sqrt{r_1})a + \sqrt{r_1}(1 - r_2)b + \sqrt{r_1}r_2c \quad (8)$$

Previously, view factor exchange matrix calculation is the main step to solve the radiative heat flux. Object geometries are meshed to small pieces so that the accuracy can be higher. Then, Monte Carlo method was implemented to get rid of using the

midpoint of the squared subarea. In the present work, multiple random variables decides the emission point.

2.1.3 Launch Direction

Ray tracing needs an emission point, ray direction and hit point. If ray emission point is selected, there should be decided an direction for tracing. In a 3-dimensional space, polar angle " θ " is determined on vertical by Equation 9 where $r3$ is the randomly selected numbers between 0 and 1.

$$\theta = \arcsin \sqrt{r3} \quad (9)$$

In Equation 10, azimuth angle " ϕ " is determined on the horizon using the $r4$ random variable between 0 and 1.

$$\phi = 2\pi r4 \quad (10)$$

All the 3-dimensional direction vector elements are calculated and are represented in vector p which can be seen in Equation.

$$p = \begin{bmatrix} \sin\theta\cos\phi \\ \sin\theta\sin\phi \\ \cos\theta \end{bmatrix}$$

2.1.4 Ray-object Intersection

Ray is traced starting from emission point through formulated direction and propagates all over the domain until seeking an intersection with another surface position. Primitive id and coordinates of the hit surface are calculated and saved. The computational calculation is highly costly task. Ray-object intersection needs many highly costly computational calculations between objects, object normals and rays as described by Glassner[1]. In order to cope with this expensive computation application,

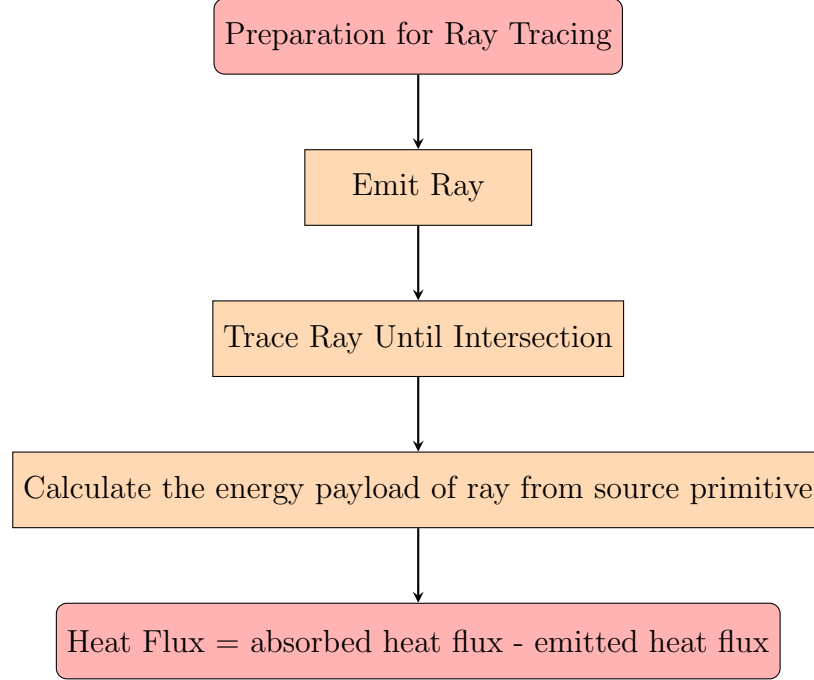
the Optix library[] is explicitly developed. To speed up this costly job, acceleration methods such as Bounding volume hierarchy, Kd-tree are applied to scenario. The path of ray bundle follows the formula in Equation 5. Parallel structure of GPU acceleration makes it available to avoid computational bottlenecks in the traditional pipeline.

2.2 Radiative Heat Flux Calculation with Monte Carlo ray tracing method in OptiX

The numerical calculation of Monte Carlo ray tracing based surface-to-surface radiation model is a non-deterministic approach. Rays are emitted from random points on the surface in random directions. Each ray carries a certain energy payload. Once the preparation for ray tracing in the OptiX side completed, rays are emitted. The scene is now capable to track real time simulation propagation. Ray tracer is executed to seek any intersection to exchange the energy payload.

Rays transfer their energies to the cells where energy is absorbed. Energy exchange with the bundle and the surface depends on the temperatures of the source and destination surface. These are supplied from the OpenFOAM boundary surface on the CPU side to GPU side.

Flow chart of the radiative heat flux algorithm



2.2.1 Algorithm for the Monte Carlo Ray Tracing based surface – to – surface radiation model

First part of the present method combined OpenFOAM and Optix starts with taking temperatures of the surfaces on CPU side and transfer the data to GPU side for calculating energy of the each ray will be emitted. Heat flux is calculated for the source cell. Rays are traced from random cells with carrying energy payload until they interact with any surface and absorbed by the surface. Otherwise, rays are reflected and diffusely send to domain for seeking an absorbing object. Then, heat flux of the cell is updated according to incoming and outgoing rays and work on GPU side is finished. This process is followed for every time iteration and each ray emitted. After that, algorithm is continued with OpenFOAM on the CPU side. Modified view factor model of chtMultiRegionFoam solver takes the heat fluxes from Optix code and assign them to related surfaces. Radiative heat flux is used as boundary condition which means source term in energy equation. By doing so, new temperatures are coming from the result of energy equation solution and OpenFOAM updates the temperatures by writing on old-temperature values. For the new iteration, updated

temperatures are transferred to Optix side again for the finding new heat flux values. It keeps doing until the iteration number or time is satisfied.

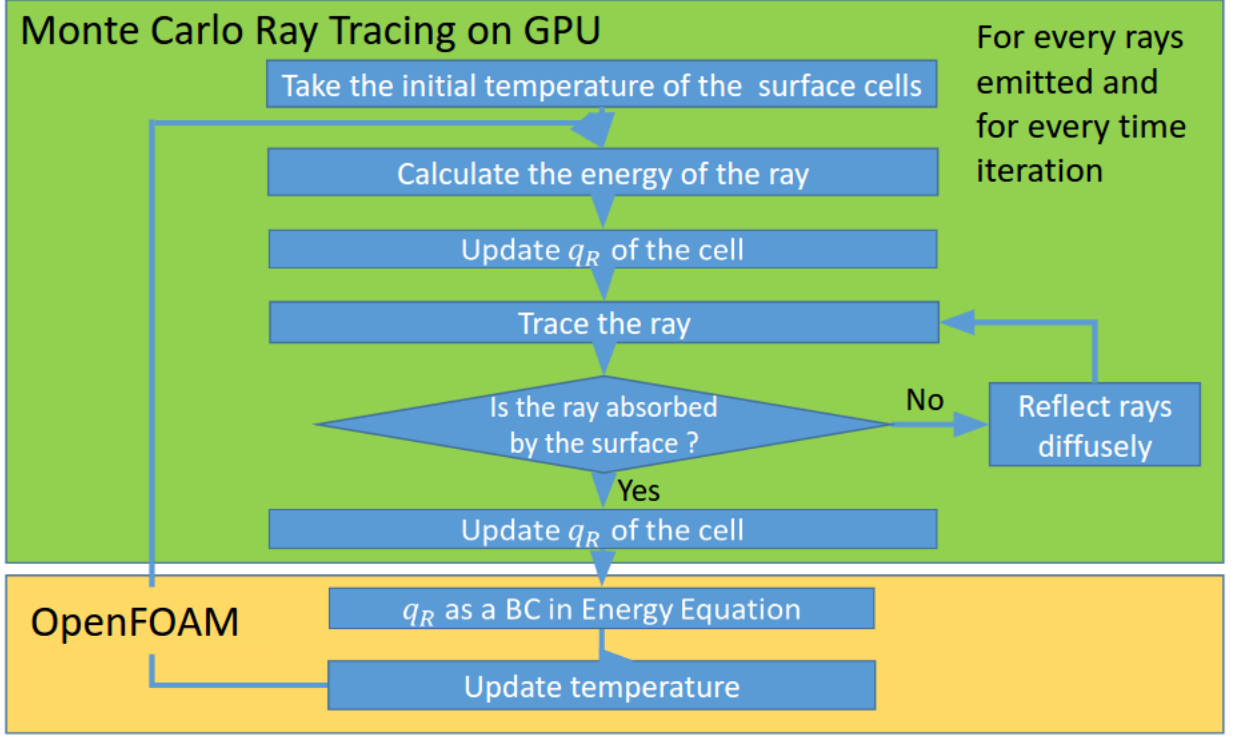


Figure 13: Algorithm for the Monte Carlo Ray Tracing based surface – to – surface radiation model

2.3 View factor Calculation – OpenFOAM View factor Calculation Utility

Rays determines the view factor as explained earlier in the previous chapter. Multiple rays are traced to achieve reliable accurate solution with Monte Carlo ray tracing method. High computation power consumption is handled with the cutting-edge graphical processors unit that supports the executing tasks easily using parallelism structure. On the other hand, OpenFOAM radiation model provides view factor using only one core for serial processing in sequential order.

As explained in the previous chapter, OpenFOAM viewFactor radiation model is

evaluating view factors between surfaces and heat exchanges by radiation. viewFactorsGen utility calculates the view factor by generating rays for every discrete surface of the patch. Radiative heat transfer calculation is using the energy transfer between ray end points that are the center of the surfaces. It makes problematic results in the critical regions which are not directly visible to each other or at the curvy shapes because utility only using the center of face. On the other hand, it is very easy for applying on the flat surfaces. Due to this standardization, there will be always areas in the complex geometries that does not benefit enough from radiative sources.

$$F_{i \rightarrow j} = \frac{1}{A_{cell, i}} \sum_{i=1}^{\left(\begin{smallmatrix} \text{number of cells} \\ \text{in plate i} \end{smallmatrix} \right)} \sum_{j=1}^{\left(\begin{smallmatrix} \text{number of cells} \\ \text{in plate j} \end{smallmatrix} \right)} \frac{\cos(\theta_i) \cos(\theta_j) A_{cell, i} A_{cell, j}}{\pi r^2}$$

Figure 14: F_{ij} exchange matrix calculation from each i surface to j surface

As described in the numerical method chapter, OpenFOAM viewFactor utility generates the exchange matrix calculating view factor from surface i to surface j and summing one by one. As seen in the Figure, terms with black color is defined for one cell i, red color is defined for the other cell j. Area of the cell i is denominator for all the summation. Both surface areas are multiplied by the cosinus angle between normals in order to find the area covering perpendicular to the area of the other plate.

CHAPTER III

RESULTS

The chapter explains combined heat transfer results of the developed applications using the Monte Carlo method in terms of effectivity, speed-up and computational cost. Hardware comparisons of various general purpose graphic processors units (GPGPU) are studied on the same test case using MCRT method. The present application developed in the research using Monte Carlo ray tracing is adjusted according to supply necessary boundary condition parameter by RTE. The results of speed-up and memory usage of OpenFOAM features and the developed solver on conjugate heat transfer scenario are discussed in this section. Convective and radiative heat transfer outcomes are also checked against between OpenFOAM and present solver.

Mesh dependency of the simulations is analyzed across view factor calculations and the relative error behaviour is explained in terms of number of cells. GPU utilization of developed method provides results that independent of mesh structure. Simulations are investigated in terms of the accuracy of the result, the computational time of the problem while checking consumption of the memory on CPU and GPU.

3.1 OpenFOAM and Monte Carlo Method View factor Calculation Utility Results

The implementation of the Monte Carlo ray tracing method is carried out according to Modest [2]. The accuracy and the speed of the developed MCRT based view factor calculator is tested on a view factor scenario between two finite plates. Objects are As shown in Figure , OpenFOAM's own view factor calculation tool carries out the calculation deterministically where as in the present method the rays are emitted from the top plate randomly and the view factor calculations are carried out based

on the statistics of the number of hitting rays on the other plate. The accuracies of both tools are compared with the analytical solution [3].

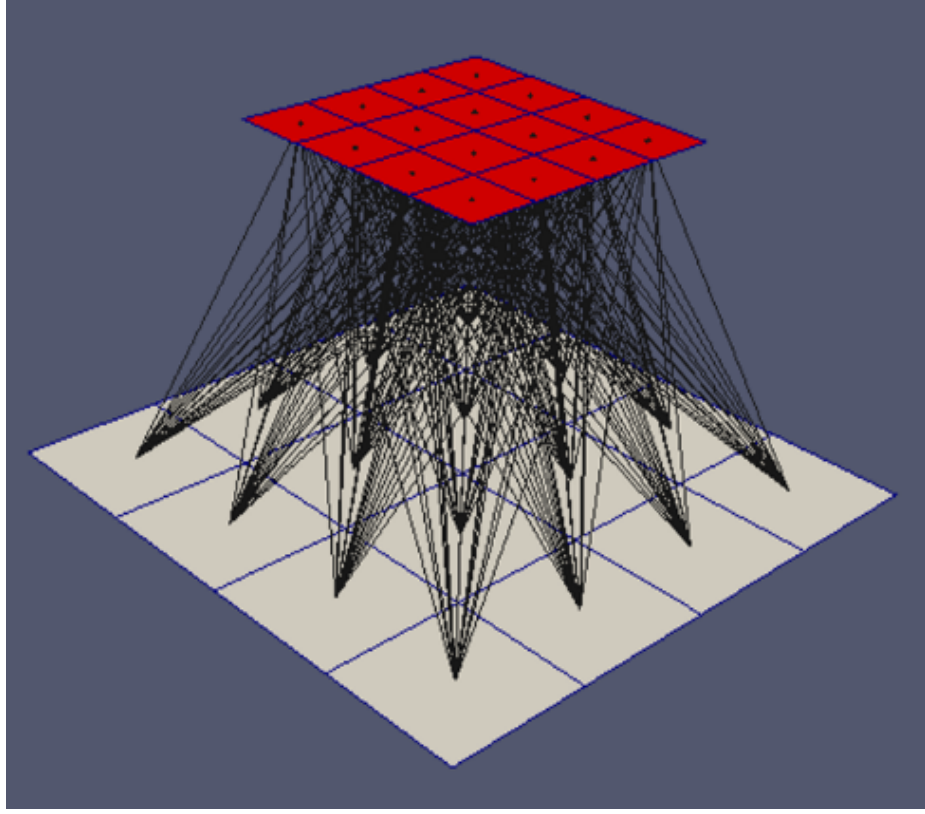


Figure 15: View factor calculations between the two planes (calculation was made by OpenFOAM software with deterministic ray tracing method).

View factor calculation between two plates given in Figure 15 occurs only at the center of the cells. There are 32 total surface cells while the 16 cells are at the bottom and top plate respectively. Each cell sends rays to the other cells to calculate the angles θ_i and θ_j . Each entry of the F_{ij} matrix is calculated by a single ray. This produces the problematic results according to changing meshes of the geometries. View factor is calculated from center no matter what the size of the cell or orientations of the cell. Specially, it leads results with high error ratio at the corners and edges.

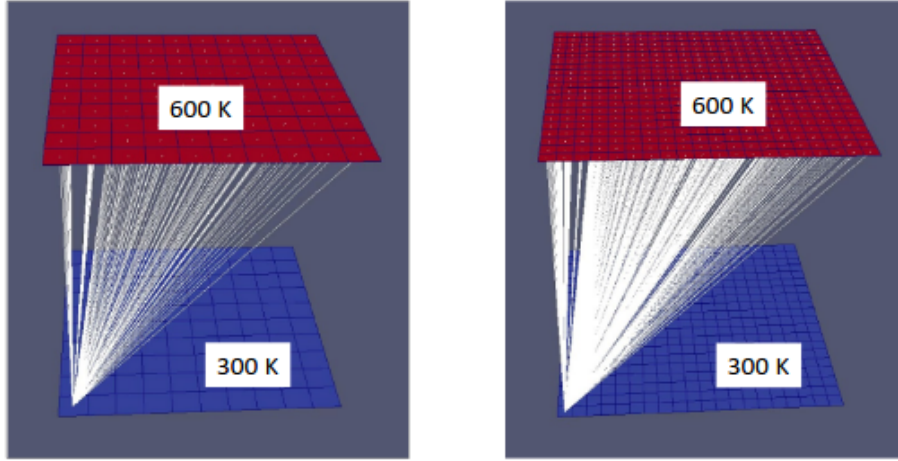


Figure 16: Calculation of Radiative Heat Flux btw surfaces : Cell Size vs. Accuracy

Openfoam utility which is responsible for view factor calculation sends rays to each surface cell of the top plate from only one cell from bottom plate as can be seen in Figure 16. Temperatures of the plates are 600K at the top and 300K at the bottom. Radiation transfer equation has been solved for these plates. Radiative heat flux accuracy gets higher as number of cells of the mesh increases. It is due to the angle separation for more number of cells. The mesh is separated small fragments and each one of them has more correct angle than previous case. By only this way, OpenFOAM may decrease its error rate. Radiation heat transfer performance is highly dependent on the mesh structure.

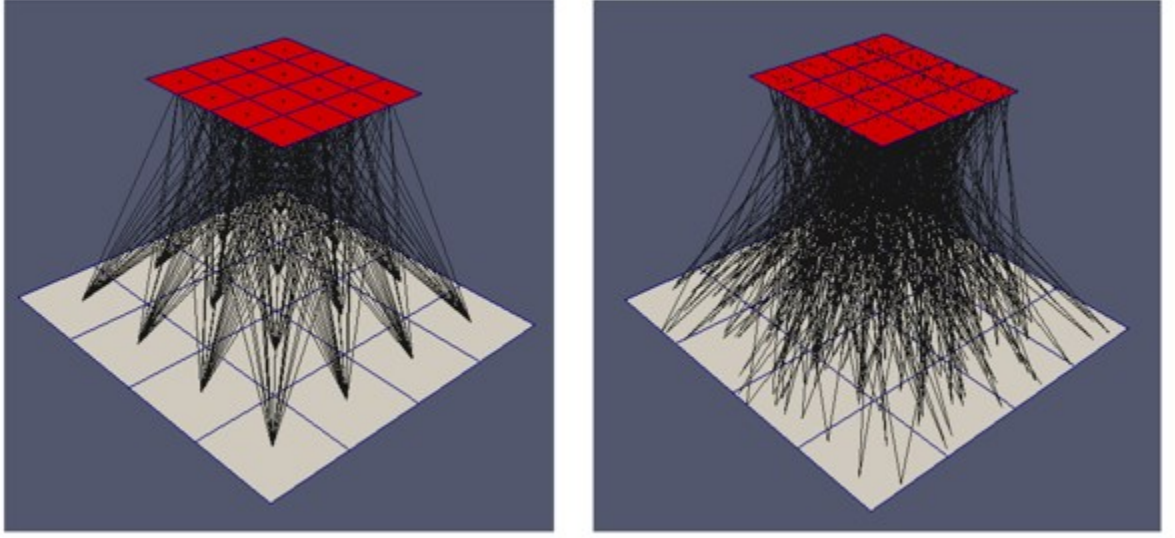


Figure 17: Figure 1: View factor calculations between the two planes (Left calculation was made by OpenFOAM software with deterministic ray tracing method. The right calculation was made by newly developed software with Monte Carlo ray tracing (MCRT) method.)

As shown in Figure 17, the accuracy of the OpenFOAM's view factor calculation tool can be increased with the increase in the number of cells on the plate. On the other hand, the accuracy of the present numerical tool is independent of the number of cells present on the plates but depends entirely on the number of rays used in the view factor calculations. Using 1 million rays, the standard deviation of the relative errors is measured as 10^{-4} which OpenFOAM view factor calculator can only reach using a plate with 64 cells in every direction.

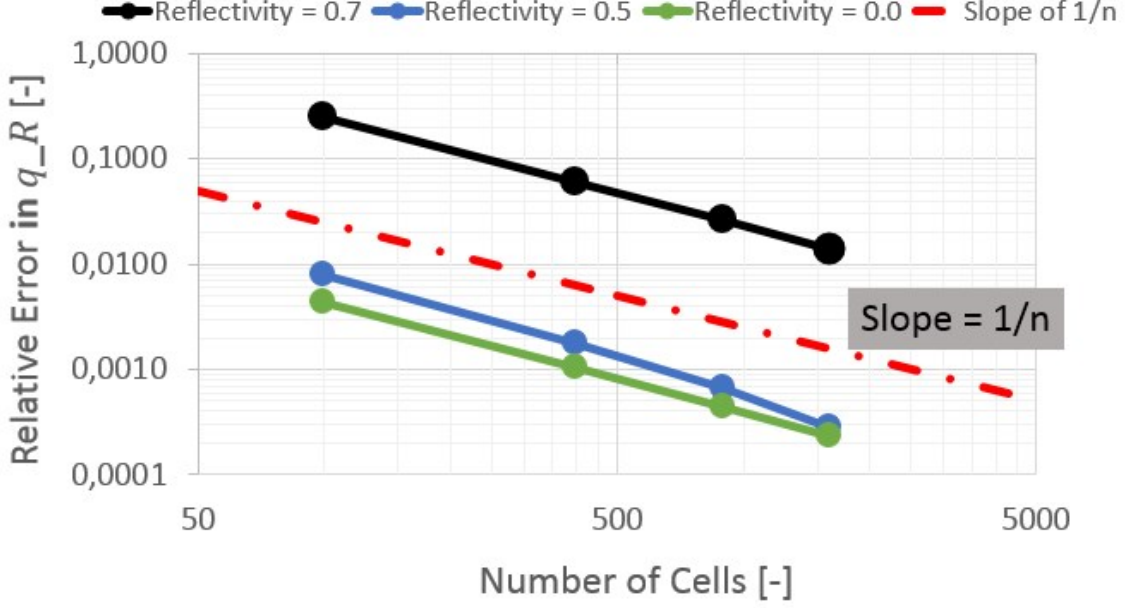


Figure 18:

OpenFOAM view factor calculation methodology also can be seen from the symptoms of relative error in q_R as total cell number increases. Several simulations have taken into account for different reflectivity values. Figure 17 displays the all error values in logarithmic scale. Reflectivity value of 0.7 provides higher relative error compared to other reflectivities. Cases with reflectivity values of 0.5 and 0.0 produces nearly same relative errors as seen in Figure 17. Apparent result of the study for relative error in q_R is that it is decreasing following the slope of $1/n$ (n is the number of cell) as the number of cells increases. Relative error in q_R scales with Δx of the cell. It is first order accurate method.

Table 4 shows the computation time for surface-to-surface radiation model in OpenFOAM. Meshes with different number of cells on the surface are used to investigate computation time. Results clearly show that surface-to-surface radiation model in OpenFOAM is dependent on number of cells. Because computational complexity is N^2 for the calculation of view factor and q_R , N^3 for the LU Decomposition

of coefficient matrix. View factor calculation time to generate F_{ij} matrix increases exponentially due to the complexity N^2 . Besides this, the most expensive part of the calculation occurs while the LU decomposition of the coefficient matrix is processed. A lot of time is spent for detailed meshes due to computational complexity N^3 . Even a small amount of increase in the number of cells produces significant rise of the time consumption.

Table 2: Computation Time for Surface-to-Surface Radiation Model in OpenFOAM

Number of Cells on the Surface	View factor Calculation	LU decomposition of the Matrix C	Computation Time of q_R per Iteration
40x40	17s	80s	1s/iter
45x45	29s	215s	2s/iter
50x50	47s	470s	2.5s/iter
Computational Complexity	N^2	N^3	N^2

Coefficient of variation of the simulations using the developed present approach with Monte Carlo method is calculated as Equation 11 where σ is standard deviation and q_R is radiative heat flux.

$$CV = \frac{\sigma}{q_R} \quad (11)$$

Figure 19 shows the coefficient of variation data for number of rays across different reflectivity values. Due to stochastic nature of Monte Carlo method, relative error is not suitable for present approach of view factor calculation. Relative standard deviation is alternative name of the coefficient of variation. Various number of rays traced using 1 million, 16 million and 64 million rays for different reflectivity values. The study carried out with reflectivity values of 0.0 and 0.5 follows similar behaviour. On the other side, simulation with reflectivity value of 0.7 has higher coefficient of variation than other reflectivity values. Importance of the simulation results is that

all the data follows the same decline ratio which is the slope of $1/\sqrt{n}$ (n is the number of rays traced). Accuracy of the heat flux scales with $1/\sqrt{n}$ as well.

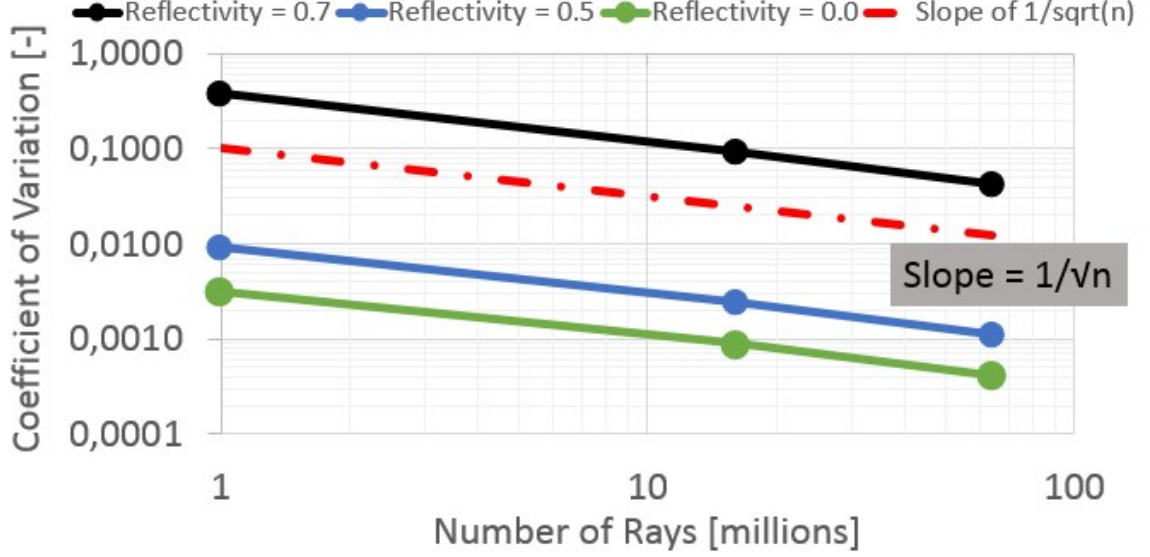


Figure 19:

Table 3 displays the computation time for heat flux using Monte Carlo ray tracing on graphic card. Simulations are processed on the Nvidia Geforce RTX 2080 8 GB on-board memory. 1, 16 and 64 million rays are traced for view factor calculation with different reflectivity values. The computation time results of Monte Carlo method are increasing as approximately proportional to $\sqrt{n}$.

Table 3: Computation Time for Heat Flux using Monte Carlo Ray Tracing on Graphic Card

Computation Time for q_R [s]			
Number of Rays Traced (millions)	Reflectivity=0.7	Reflectivity=0.5	Reflectivity=0.0
1	0.015	0.015	0.015
16	0.060	0.049	0.049
64	0.188	0.169	0.132

3.2 *Transient Conjugate Heat Transfer Scenario*

The developed surface-to-surface radiation model has been applied to transient conjugate heat transfer scenario in Figure 21. "chtMultiRegionMultiFoam" solver of OpenFOAM has been used for conduction and convection heat transfer. Radiation part is changed with the developed radiation model on Optix.

Scenario involves multi-region as air and glass solid. A glass object is placed inside a hexahedral enclosure with the sides. The air region consists of 700K tetrahedral cells and glass solid region consists of 30K tetrahedral cells. Boundary condition temperature for right plate is 600°K while all the temperature of remaining mesh is 300°K. Air flows from inlet with 1m/s speed and 300° temperature. Radiation heat transfer occurs between glass-to-glass and glass-to-walls.

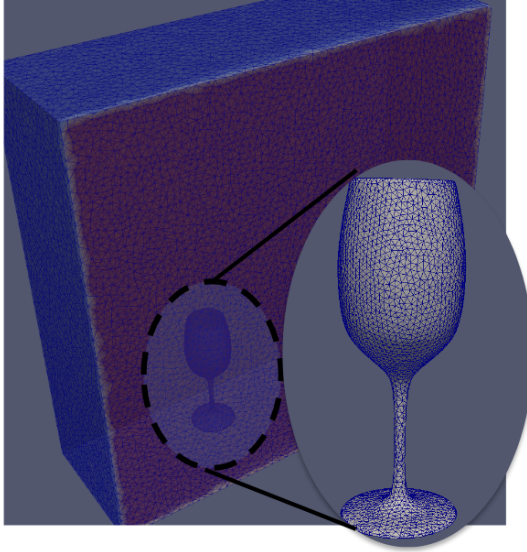


Figure 20:

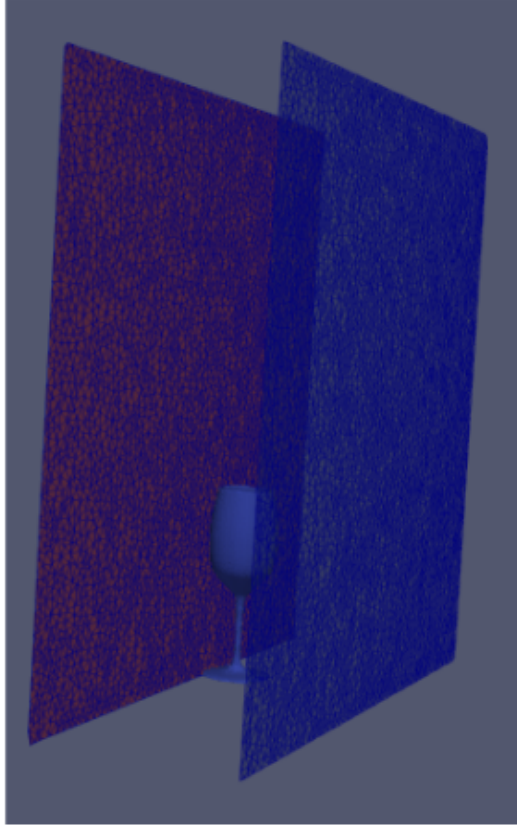


Figure 21:

The `chtMultiRegionFoam` solver is taking into account each region separately. Boundary condition couples regions by determining heat flux of the surfaces.

In the study, boundary condition is an important parameter for the air-solid heat transfer interaction. Temperature values on the both sides of the patch are same for the mapped surfaces. OpenFOAM's "`compressible::turbulentTemperatureRadCoupledMixed`" has been selected for the boundary condition on the wineglass-air interaction.

Heating process simulation has evaluated for the case of clock time 100 seconds. Computation cost of solution of the current scenario involves multiple modes of the heat transfer mechanisms in furnace. Convection and conduction heat modes are solved using open-source OpenFOAM solver while the radiation mode is calculated

Table 4: Computational Cost for Radiation Heat Transfer in OpenFOAM using 1 CPU

Computation Time for q_R [s]			
Total Number of Surface Cells (Glass + Walls of the Furnace)	View Factor Calculation	LU Decomposition of the Matrix C	
19720	12min	136min	0.36min/iter
	Only once	Only once	Every Iteration 717 mins for 2000 iteration

using developed solver with the Monte Carlo method. A wineglass solid mesh consists of 30K cells. The transient conduction problem inside the solid for single glass is solved only on the single CPU core. The steady convection is solved for the air inside of all the furnace consisting of mesh size of 750K cells. Convection heat transfer problem is suitable for the multiple CPU usage. In order to accelerate the solution, different number of CPU's up to 28 cores are tested. Concurrently, surface-to-surface radiation heat has been solved by in-house developed software of the study utilizing only a single the NVIDIA TITAN Xp graphic card.

The developed in-house present radiation software has solved the radiation mode only. Surface-to-surface radiation exchange happens between a hot wall (source) and glass (target). In OpenFOAM software, exchange matrix has to be calculated in the pre-process of solving to start the view factor based radiation. Mesh characteristic is hugely effective in the accuracy of the view factor exchange. Computation time also increases while the mesh complexity evolves. Due to the limitation of memory allocation of OpenFOAM's view factor calculation, exchange matrix of every mesh can not be calculated successfully. On the other hand, present approach with OptiX SDK has not much limitations considering time or memory to calculate view factor.

Main calculation subject of surface-to-surface Monte Carlo method is a relation of a ray between intersection points and boundaries which are a large set of surfaces. The radiation transport scenario between glass and hot wall has been tested, where the

temperature of whole environment is 300K except the 600K fixed hot wall. 3D Glass boundary has portioned to 12590 face elements and 2D wall has 7130 face elements. Result of view factor on the glass in scaled version can be seen in Figure 22. 1 million rays are traced for this case but accuracy increases as more rays are traced. However, standard deviation of view factor for 100 time step fluctuates maximum 0.06 at the non-central faces to the wall. More rays can be traced to get more accurate results but there are already several iterations are held and each one of them is tracing 1 million rays. The view factor calculation with OptiX is more close to reality with smooth transition from maximum to minimum in comparison to face center-to-center OpenFOAM view factor calculation where face normals directly effects the accuracy. Even the neighbour faces on the glass can get inconsistent values. This directly effects the heat transfer exchange calculation in bad manner. But, stochastic method is able to perform better heat exchange calculation and distribution over the glass.

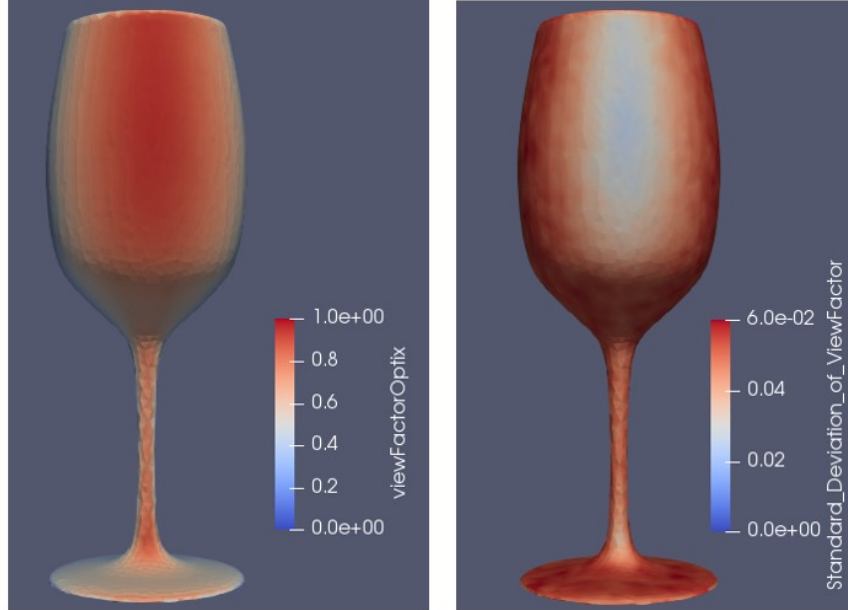


Figure 22: View Factor between the glass and the heater (Left).Standard deviation in view factor calculation (Right)

Total computation time comparison can be seen in Figure 23. Simulations are between standard OpenFOAM on 1 CPU and GPU accelerated radiation solver for OpenFOAM on 1 CPU + 1 GPU. Conduction and convection part takes similar time for both scenario solving on 1 CPU core. Most distinctive parameter for radiation heat transfer is now that the one using parallel structure of GPU instead of using serial pipeline structure of CPU. Radiation part specially gained significant 80x speed-up for current scenario with the aid of GPU performance. More complex scenes and geometries can be accelerated further compared to OpenFOAM.

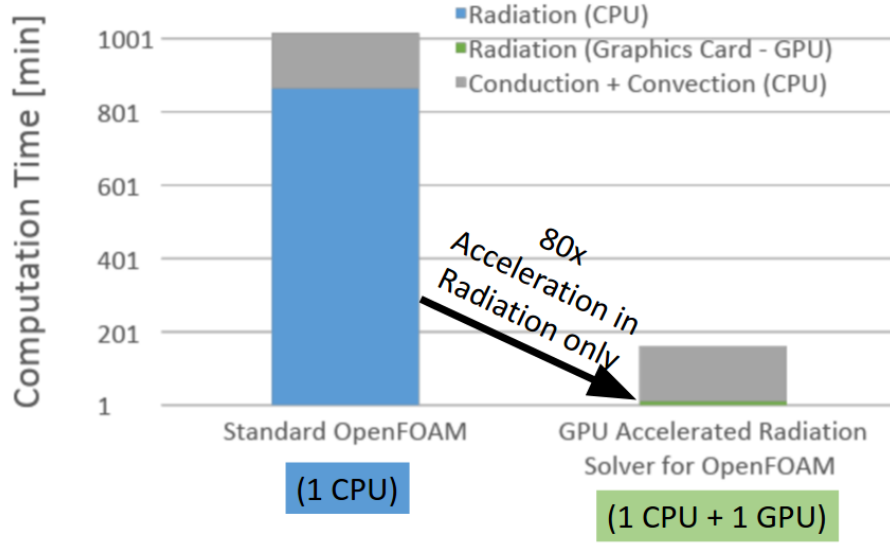


Figure 23: Total computation time comparison

Total computation time comparison in Figure 24 is held the simulations using multiple CPUs for conduction and convection heat transfer while the radiation mode is solved on the developed solver on single GPU. CPU based solvers are experiencing proportional speed-up as number of CPU cores are used. More processors do not always mean faster execution after a specific treshold where the data transfer cost is a burden.

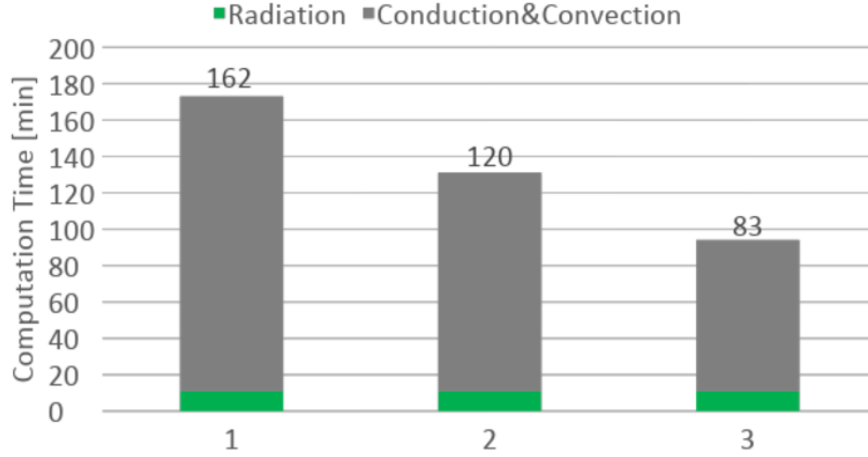


Figure 24: Total Computation Time for using Multiple CPUs and single GPU

OpenFOAM conjugate heat transfer multi-region solver can be solved in parallel processing using multiple number of CPU cores. Simulations in Figure 25 are evaluated using the built-in chtMultiRegionFoam solver of OpenFOAM on up to 28 CPU cores. Conduction and convection heat transfer modes are solved faster with multiple CPU cores. However, radiation heat transfer does not follow time saving trend with parallelism. Heat flux calculation takes longer time with more CPU cores due to data transfer cost between processors. At the end, when overall simulation time is considered, there are not significant difference between executing conjugate heat transfer solver on single CPU core or on 28 cores. As a result, OpenFOAM surface-to-surface radiation solver is not parallelable.

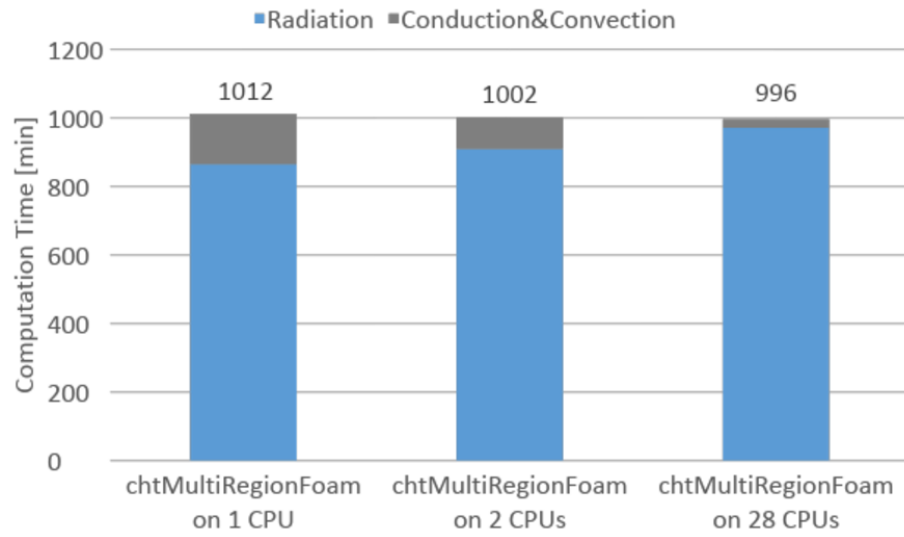


Figure 25:

CHAPTER IV

CONCLUSION

- A graphics card accelerated surface-to-surface radiation model using Monte Carlo method has been developed. The developed MCRT based view factor calculator is integrated into the OpenFOAM framework by replacing OpenFOAM's own view factor model.
- The surface – to – surface radiation model in the OpenFOAM solver is not suited to large scale simulations.
- Developed solver is merged with OpenFOAM's built-in conjugate heat transfer solver "chtMultiRegionFoam". The GPU acceleration impact on the total simulation time of the solver on the multi-physics scenarios has been analyzed and compared with conventional CPU approach. Using the modern graphic processors, computational cost of the radiation calculation significantly improved.
- For the scenario conducted in the study, high-end GPU provides the performance advantage of nearly 80x speed-up radiation only in comparison to multi-core high-end CPU. With the help of GPU utilization, the NVIDIA Optix SDK allows the demonstration of the results on the scene simultaneously with the calculation has been done in each iteration step.
- Apart from 80x acceleration in radiation, the overall run time of the conjugate heat transfer solver in the selected scenario is around 7x speed-up.
- Developed solver is not mesh dependent like OpenFOAM's view factor radiation model, error of the radiation heat transfer outcomes per mesh face does not vary dramatically with the cell size.

- The accuracy of the solver has been verified on a radiative transfer scenario between parallel plates for different reflectivities. Higher accuracy on surfaces with low reflectivity.
- The GPU based solver is almost 2 orders of magnitude faster in the studied conjugate heat transfer scenario covering 20k surface cells.
- In this study, the GPU computing power showed tremendous opportunities of the applying ray tracing numerical methods to solve the radiative heat transfer equation.

CHAPTER V

ACKNOWLEDGEMENT

- This project has been funded by Scientific and Technological Research Council of Turkey (TÜBİTAK) Project No : 117M420.
- We also acknowledge the support of NVIDIA Corporation with the donation of the Titan Xp GPU used for this research. Partial support has also been received from the Center of Energy, Environment and Economy at Ozyegin University.

APPENDIX A

SOME ANCILLARY STUFF

Ancillary material should be put in appendices, which appear just before the bibliography.

Bibliography

- [1] D. E. Knuth, *The T_EX Book*. Reading, Massachusetts: Addison-Wesley, 1984. Reprinted as Vol. A of *Computers & Typesetting*, 1986.
- [2] D. E. Knuth, *T_EX: The Program*, vol. B of *Computers & Typesetting*. Reading, Massachusetts: Addison-Wesley, 1986.
- [3] D. E. Knuth, “The WEB system for structured documentation, version 2.3,” Tech. Rep. STAN-CS-83-980, Computer Science Department, Stanford University, Stanford, California, Sept. 1983.
- [4] D. E. Knuth, “Literate programming,” *The Computer Journal*, vol. 27, pp. 97–111, May 1984.
- [5] D. E. Knuth, “A torture test for T_EX, version 1.3,” Tech. Rep. STAN-CS-84-1027, Computer Science Department, Stanford University, Stanford, California, Nov. 1984.
- [6] R. K. Furuta and P. A. MacKay, “Two T_EX implementations for the IBM PC,” *Dr. Dobbs’s Journal*, vol. 10, pp. 80–91, Sept. 1985.
- [7] J. Désarménien, “How to run T_EX in french,” Tech. Rep. SATN-CS-1013, Computer Science Department, Stanford University, Stanford, California, Aug. 1984.
- [8] A. L. Samuel, “First grade T_EX: A beginner’s T_EX manual,” Tech. Rep. SATN-CS-83-985, Computer Science Department, Stanford University, Stanford, California, Nov. 1983.
- [9] L. Lamport, *ΛT_EX: A Document Preparation System. User’s Guide and Reference Manual*. Reading, Massachusetts: Addison-Wesley, 1986.
- [10] M. D. Spivak, *The Joy of T_EX*. American Mathematical Society, 1985.
- [11] O. Patashnik, *BibT_EXing*. Computer Science Department, Stanford University, Stanford, California, Jan. 1988. Available in the BibT_EX release.
- [12] O. Patashnik, *Designing BibT_EX Styles*. Computer Science Department, Stanford University, Jan. 1988.
- [13] D. Fuchs, “The format of T_EX’s DVI files version 1,” *TUGboat*, vol. 2, pp. 12–16, July 1981.
- [14] D. Fuchs, “Device independent file format,” *TUGboat*, vol. 3, pp. 14–19, Oct. 1982.

VITA

You write your vita here.